

functions

What is a function?



- A function is a set of statements that take inputs, do some specific computation and produces output.
- The idea is to put some commonly or repeatedly done task together and make a function.
- so that instead of writing the same code again and again for different inputs, we can call the function
- Functions are defined with the def statement:

```
#function definition
def function_name(parameters)    //formal parameters

    // body of the function

#function call
function_name(parameters)    //actual paramters
```

Function - Example



A simple Python function to add two numbers

```
def sumofnumbers(x,y ):
    s=x+y
    return s
```

program code

```
Sum_ = sumofnumbers(3,4)
print (Sum_)
```

Output:
7

invoke a function by writing the function name followed by a tuple of function arguments

Function - Example



**# A simple Python function to check
whether x is even or odd**

```
def evenOdd( x ):  
    if (x % 2 == 0):  
        print "even"  
    else:  
        print "odd"
```

program code

```
evenOdd(2)  
evenOdd(3)
```

Output:
even
odd

invoke a function by writing the function name followed by a tuple of function arguments

Pass by Reference or pass by value?



- ✓ In Python every variable name is a reference.
- ✓ When we pass a variable to a function, a new reference to the object is created. Parameter passing in Python is same as **reference passing** in Java

Here x is a new reference to same list lst

```
def myFun(x):
```

```
    x[0] = 20
```

program Code (Note that lst is modified

after function call.

```
lst = [10, 11, 12, 13, 14, 15]
```

```
myFun(lst);
```

```
print(lst)
```

Output :

```
[20, 11, 12, 13, 14, 15]
```

Parameter Passing



```
def myFunc(p):  
    p=10
```

```
# Driver code
```

```
x = 2
```

```
myFunc(x)
```

```
print(x)
```

If reference is assigned to something else from function, then the connection between passed and recieved reference is broken.

Output :

2

Parameter Passing and Return Values

- if a mutable object (such as a list or dictionary) is passed to a function where it's then modified, those changes will be reflected in the original object
- The **return statement** returns a value from a function.
- If no value is specified or you omit the return statement, the None object is returned.
- To return multiple values, place them in a tuple

Function return value



```
def sum_(a,b):  
    return (a+b)
```

```
s=sum_(2,3)  
print(s)
```

OUTPUT
5

```
def division(a,b):  
    q = a//b  
    r = a%b  
    return (q, r)
```

```
x,y = division(10,2)  
print(x,y)
```

OUTPUT
x is 5
y is 0

Types of arguments of function



- **Default arguments**
- **Keyword arguments**
- **Variable length argument**
- **** argument**

Default arguments:

- A default argument is a parameter that assumes a default value if a value is not provided in the function call for that argument

```
# Python program to demonstrate
# default arguments
def myFun(x, y=50):
    print("x: ", x)
    print("y: ", y)

# Driver code (We call myFun() with only one argument)
myFun(10)
```

Output:
x: 10
y: 50

Keyword arguments



- Function arguments can also be supplied by explicitly naming each parameter and specifying a value. These are known as **keyword arguments**

```
# Python program to demonstrate Keyword arguments
def exFunc(w,x,y,z):
    Statements
```

```
# Keyword argument invocation
exFunc(x=3, y=22, w='hello', z=[1,2])
```

```
exFunc('hello', 3, z=[1,2], y=22)
```

```
exFunc(3, 22, w='hello', z=[1,2]) # TypeError. Multiple values for w
```

Variable length argument



- A function can accept a any number of parameters if an **asterisk (*)** is added to the last parameter name.

```
# Python program to illustrate  
# *args for variable number of arguments  
def myFunc(n,*argv):  
    print("number of strings passed is",n)  
    for arg in argv:  
        print (arg)
```

```
myFunc(4,'Hello', 'Welcome', 'to', 'Python')
```

```
Output:  
Number of string passed is 4  
Hello  
Welcome  
to  
Python
```

** argument



- If the last argument of a function definition begins with **, all the additional keyword arguments (those that don't match any of the other parameter names) are placed in a dictionary and passed to the function

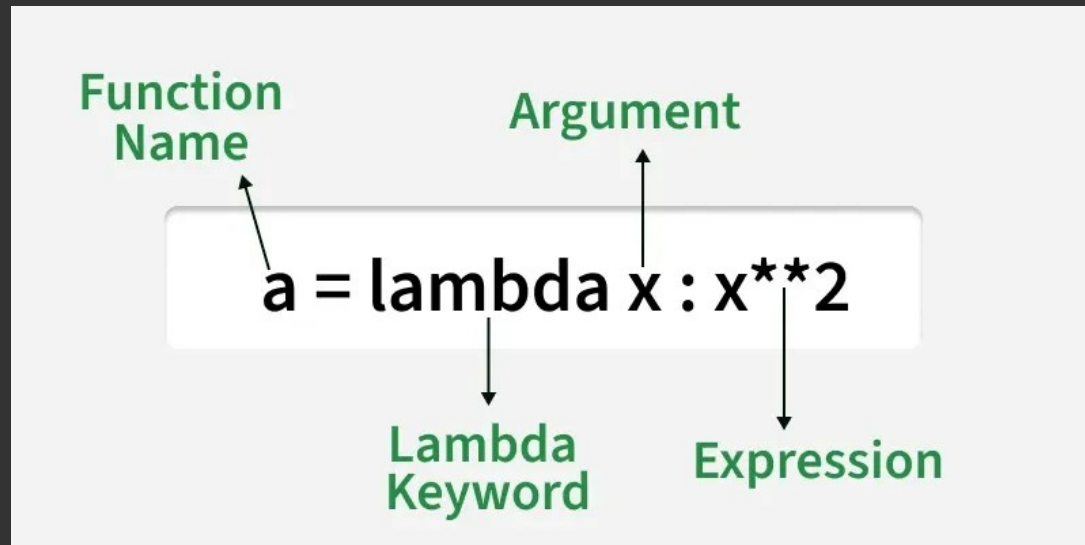
```
def myfunc(n,**parm):  
    print("n is %d"%(n))  
  
    for key in parm:  
        print("%s - %s"%(key,parm[key]))
```

```
myfunc(5,fgcolor="white",width=5, border=1)
```

Lambda Functions



- A lambda function is a small anonymous function.
- A lambda function can take any number of arguments, but can only have one expression.
- Result of that expression is returned automatically (no return keyword needed).
- Lambda functions are created using the **lambda** keyword. Below is the syntax:



```
sum_ = lambda a : a + 10  
print(sum_(5))
```

Lambda Functions



- Multiply argument a with argument b and return the result:

Example

```
x = lambda a, b : a * b  
print(x(5, 6))
```

```
x = lambda a, b, c : a + b + c  
print(x(5, 6, 2))
```

Lambda with Built-in Functions



- Lambda functions are commonly used with built-in functions like **map()**, **filter()**, and **sorted()**.

Using Lambda with map() - The map() function applies a function to every item in an iterable:

```
numbers = [1, 2, 3, 4, 5]

doubled = list( map( lambda x: x * 2, numbers) )

print(doubled)
```

OUTPUT

[2, 4, 6, 8, 10]

Lambda with Built-in Functions



- **Using Lambda with filter()** - The filter() function creates a list of items for which a function returns True:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]

odd_numbers = list(filter(lambda x: x % 2 != 0, numbers))

print(odd_numbers)
```

OUTPUT

[1, 3, 5, 7]

Lambda with Built-in Functions



- **Using Lambda with sorted()** - The sorted() function can use a lambda as a key for custom sorting:

Sorting based on length of the words

```
words = ["apple", "pie", "banana", "cherry"]  
  
sorted_words = sorted(words, key=lambda x: len(x))  
  
print(sorted_words)
```

OUTPUT

```
['pie', 'apple', 'banana', 'cherry']
```